Systemová príručka

Táto príručka slúži ako technická sprievodná dokumentácia k implementovanému riešeniu a vytvára rámec pre detailnejší opis jednotlivých častí systému. Nasledujúce sekcie postupne rozoberajú účel príručky, charakteristiku systému, architektúru, konfiguráciu, dátový model, rozhrania aj bezpečnostné mechanizmy.

Účel a rozsah kapitoly

Táto kapitola predstavuje technickú príručku k demonštračnej implementácii viacfaktorovej autentifikácie pre firemný portál. Text je spracovaný priamo na základe zdrojového kódu, konfigurácie a prítomných prevádzkových artefaktov v analyzovanom repozitári v stave z 22. apríla 2026.

Príručka slúži ako technická dokumentácia k odovzdanému riešeniu. Jej cieľom je:

- zhrnúť architektúru systému a hranice jednotlivých modulov,
- opísať technologickú skladbu a prevádzkové požiadavky,
- zdokumentovať perzistentné dáta, API rozhrania a autentizačné toky,
- pomenovať bezpečnostné mechanizmy a známe obmedzenia implementácie.

Charakteristika systému

Riešenie implementuje viacfaktorovú autentifikáciu, ktorá kombinuje:

- znalosť hesla,
- lokálne zaregistrované zariadenie,
- biometrické overenie odtlačkom prsta,
- postkvantový digitálny podpis.

Pri registrácii používateľa sa vytvorí OPAQUE registračný záznam. Následne sa na lokálnom zariadení zaregistruje fingerprint šablóna a vygeneruje sa pár kľúčov algoritmu ML-DSA-44. Pri prihlasovaní backend najprv overí heslo cez OPAQUE protokol, potom vydá jednorazovú výzvu a lokálny agent ju po úspešnom biometrickom overení podpíše. Až po úspešnej verifikácii podpisu backend vydá finálny JWT token.

Nasledujúca tabuľka sumarizuje základné vrstvy implementovaného riešenia, technológie použité v každej vrstve, ich predvolené adresy a praktickú úlohu v systéme. Ide o rýchly orientačný prehľad komponentov, z ktorých sa skladá celé demonštračné riešenie.

Tabuľka 1: Prehľad hlavných vrstiev systému a ich úloh

Vrstva	**Technológie**	**Predvolená adresa**	**Úloha**
Frontend	React, TypeScript, Vite, klientská časť OPAQUE	`https://127.0.0.1:5173`	Webové rozhranie pre registráciu, prihlásenie a zobrazenie dashboardu.
Backend	Node.js, Express, TypeScript, SQLite, OPAQUE, JWT	`https://127.0.0.1:4000`	Riadenie autentizačného toku, správa používateľských záznamov, vydávanie challenge a overenie druhého faktora.
Lokálny agent	Python, Flask, pyserial, pqcrypto, PyCryptodome	`https://127.0.0.1:5555`	Komunikácia so senzorom AS608, registrácia odtlačku, generovanie a ochrana PQC kľúčov, podpisovanie challenge.
Pomocné skripty	Python	lokálne bez samostatného portu	Verifikácia ML-DSA podpisu a experimentálne biometrické merania.

Štruktúra projektu

Z technického hľadiska sú rozhodujúce najmä tieto časti repozitára:

```
client/
  src/
    App.tsx
    main.tsx
    components/ProtectedRoute.tsx
    pages/Login.tsx
    pages/Dashboard.tsx
  package.json
  vite.config.ts
  tsconfig.json

server/
  src/
    server.ts
    metrics.ts
  package.json
  tsconfig.json
  users.db
  server_setup.txt
  tls/

tools/
  pqc_verify.py
  fingerprint/
    agent.py
```

```
as608_ops.py
metrics.py
requirements.txt
run_frr_pilot.py
logs/
pqc_keys/
```

Adresáre `node_modules/`, `dist/` a `__pycache__/` sú odvodené artefakty vytvorené inštaláciou alebo build procesom a nie sú predmetom tejto technickej analýzy.

Architektúra riešenia

Táto sekcia rozdeľuje implementované riešenie na hlavné architektonické časti a stručne vysvetľuje ich zodpovednosti. Samostatne sú opísané frontend, backend, lokálny fingerprint agent a pomocné skripty, ktoré zabezpečujú doplnkové prevádzkové a experimentálne úlohy.

Frontend

Frontendová časť je implementovaná v adresári `client/`. Vstupným bodom aplikácie je súbor `main.tsx`, ktorý vyrenderuje komponent `App.tsx`. Ten definuje dve hlavné routy:

- úvodnú prihlasovaciu obrazovku,
- dashboard dostupný cez komponent `ProtectedRoute`.

Komponent `Login.tsx` obsahuje celý používateľský tok:

- registráciu hesla cez OPAQUE,
- registráciu zariadenia cez lokálneho agenta,
- prihlásenie cez OPAQUE,
- získanie challenge,
- odoslanie challenge na podpis agentovi,
- odovzdanie podpisu backendu na finálne overenie.

Komponent `Dashboard.tsx` po prihlásení načítava zo servera zoznam používateľov z databázy a zobrazuje ho v prehľadovej tabuľke.

Backend

Backendová logika je sústredená v súbore `server/src/server.ts`. Server:

- obsluhuje OPAQUE registráciu a prihlasovanie,
- spravuje SQLite databázu používateľov,
- ukladá verejné PQC kľúče používateľov,

- generuje challenge pre druhý faktor,
- verifikuje podpisy vytvorené lokálnym agentom,
- zaznamenáva výkonové a prevádzkové metriky.

Server je publikovaný cez HTTPS a vynucuje TLS 1.3. Pri štarte automaticky inicializuje databázovú schému a v prípade potreby dopĺňa chýbajúce stĺpce pomocou migračnej logiky v kóde.

Lokálny fingerprint agent

Agent implementovaný v `tools/fingerprint/agent.py` predstavuje samostatnú lokálnu službu. Jeho úlohou je:

- komunikovať so senzorom AS608 cez sériové rozhranie,
- zaregistrovať používateľský odtlačok do internej pamäte senzora,
- vygenerovať pár kľúčov algoritmu ML-DSA-44,
- šifrovane uložiť súkromný kľúč a medzikľúč KEK,
- po biometrickom overení podpísať challenge od backendu.

V tomto modeli je súkromný kľúč uložený len lokálne. Backend uchováva iba verejný kľúč a nikdy nedisponuje materiálom potrebným na vytvorenie podpisu.

Pomocné skripty

Súbor `tools/pqc_verify.py` realizuje verifikáciu ML-DSA podpisu v prostredí Python. Backend ho volá synchronne z Node.js procesu, pretože použitá PQC knižnica je dostupná v Python ekosystéme.

Súbor `tools/fingerprint/run_frr_pilot.py` je pomocný skript pre pilotné meranie biometrickej charakteristiky FRR. Na komunikáciu používa endpoint `/biometric/probe` lokálneho agenta a zaznamenáva výsledky do CSV súboru.

Technologická skladba a systémové požiadavky

Nasledujúca časť sumarizuje technológie použité v jednotlivých vrstvách systému a zároveň vymedzuje základné prevádzkové predpoklady. V prvej podsekcii sú uvedené použité technológie a v druhej podsekcii podmienky, ktoré musí prostredie spĺňať pred nasadením riešenia.

Použité technológie

- frontend: React 18, React Router, TypeScript, Vite, `@serenity-kit/opaque`,
- backend: Node.js, Express, TypeScript, better-sqlite3, jsonwebtoken, `@serenity-kit/opaque`,
- agent: Python, Flask, flask-cors, requests, pqcrypto, pycryptodome, pyserial, psutil,
- hardvérová vrstva: fingerprint senzor AS608 komunikujúci cez sériový port.

Prevádzkové predpoklady

Správna prevádzka systému predpokladá:

- Node.js 20 alebo novší,
- Python 3.11 alebo 3.12,
- dostupný sériový port, napríklad `COM3`,
- lokálne TLS certifikáty akceptované v prehliadači aj v Node.js procese,
- fyzicky pripojený fingerprint senzor AS608 pri biometrických operáciách.

Nasadenie a spustenie

Táto sekcia opisuje praktický postup uvedenia systému do prevádzky. Najprv zhŕňa inštaláciu závislostí a následne odporúčané poradie štartu jednotlivých služieb tak, aby bolo možné riešenie korektne spustiť.

Inštalácia závislostí

Odporúčané príkazy na inštaláciu závislostí sú:

```
cd client
npm install

cd server
npm install

cd tools/fingerprint
py -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Vo Windows PowerShelli môže byť potrebné namiesto `npm` použiť `npm.cmd`.

Poradie štartu služieb

Komponenty sa odporúča spúšťať v tomto poradí:

1. backend server,
2. lokálny fingerprint agent,
3. frontendová aplikácia.

Typické spúšťacie príkazy:

```
cd server
npm run dev

cd tools/fingerprint
.venv\Scripts\python.exe agent.py --host 127.0.0.1 --port 5555
```

```
cd client
npm run dev
```

Po úspešnom štarte sú komponenty dostupné na adresách `https://127.0.0.1:4000`, `https://127.0.0.1:5555` a `https://127.0.0.1:5173`.

Konfigurácia systému

V tejto časti sú zhrnuté najdôležitejšie konfiguračné parametre jednotlivých komponentov riešenia. Podsekcie samostatne uvádzajú nastavenia frontendu, backendu a lokálneho agenta.

Frontend

Frontend používa najmä tieto premenné:

- `VITE_API_BASE`, predvolene `/api`,
- `VITE_AGENT_BASE`, predvolene `https://127.0.0.1:5555`.

Backend

Backend používa najmä tieto premenné:

- `JWT_SECRET`, predvolená hodnota `dev-secret-key`,
- `PYTHON_EXE`, voliteľná explicitná cesta k Python interpretru,
- `TLS_CERT` a `TLS_KEY` pre vlastné cesty k TLS materiálu,
- `METRICS_ENABLED` a `METRICS_PATH` pre zber metrík.

Lokálny agent

Agent používa najmä tieto premenné:

- `API_BASE`, predvolene `https://127.0.0.1:4000/api`,
- `FRONTEND_ORIGIN`, predvolene `https://127.0.0.1:5173`,
- `METRICS_ENABLED` a `METRICS_PATH` pre agentové metriky.

Perzistentné dáta a dátový model

Nasledujúca sekcia vysvetľuje, aké údaje systém dlhodobo uchováva a aký majú význam pre prevádzku riešenia. Osobitne sa venuje databáze backendu a súborovým artefaktom vytváraným mimo nej.

SQLite databáza

Backend používa databázový súbor `server/users.db`. Kľúčové tabuľky sú `users` a `pqc_challenges`.

Tabuľka users

Nasledujúca tabuľka opisuje štruktúru hlavnej používateľskej tabuľky `users`. Uvádza stĺpce, ktoré sú dôležité pre identitu používateľa, uloženie OPAQUE registračného záznamu aj evidenciu verejného kľúča a čítača podpisu pre druhý faktor.

Tabuľka 2: Štruktúra databázovej tabuľky `users`

Stĺpec	**Typ/logika**	**Význam**
`username`	primárny kľúč	Prihlasovacie meno používateľa.
`userIdentifier`	text	Interný identifikátor používateľa pre OPAQUE server.
`registrationRecord`	text	OPAQUE registračný záznam používateľa.
`bio_helper_index`	voliteľný integer	Pomocný index pre alternatívny biometrický tok.
`bio_helper_data`	voliteľný text	Pomocné biometrické dáta pre alternatívny tok.
`bio_key_hash`	voliteľný text	Hash biometrického kľúča pre alternatívny tok.
`pqc_public_key`	voliteľný text	Verejný PQC kľúč v Base64 podobe.
`pqc_algorithm`	voliteľný text	Názov algoritmu, aktuálne `ML-DSA-44`.
`pqc_created_at`	voliteľný text	Čas registrácie verejného kľúča.
`pqc_sign_count_last`	integer	Posledný serverom akceptovaný čítač podpisu.

Tabuľka pqc_challenges

Nasledujúca tabuľka zachytáva štruktúru tabuľky `pqc_challenges`, ktorá slúži na správu jednorazových výziev pri druhom faktore. Jej obsah je dôležitý najmä pre kontrolu expirácie výzvy a ochranu proti opakovanému prehratiu.

Tabuľka 3: Štruktúra databázovej tabuľky `pqc_challenges`

Stĺpec	**Typ/logika**	**Význam**
`id`	primárny kľúč	UUID identifikátor challenge.
`username`	text	Používateľ, pre ktorého bola challenge vydaná.
`challenge`	BLOB	Náhodná 32-bajtová výzva uložená v binárnej forme.
`expires_at`	integer	Čas expirácie výzvy v milisekundách od epochy.
`used`	integer	Príznak, či už bola challenge použitá.

Súborové artefakty

Mimo databázy systém pracuje aj s nasledujúcimi súbormi:

- `server/server_setup.txt`: dlhodobý parameter OPAQUE servera,
- `server/tls/*`: TLS certifikát, kľúč a root CA pre backend,
- `tools/fingerprint/tls/*`: TLS materiál lokálneho agenta,
- `tools/fingerprint/pqc_keys/device_master.key`: lokálny koreňový kľúč zariadenia,
- `tools/fingerprint/pqc_keys/<user>_priv.enc`: šifrovaný súkromný kľúč používateľa,
- `tools/fingerprint/pqc_keys/<user>_kek.enc`: šifrovaný KEK používateľa,
- `tools/fingerprint/pqc_keys/<user>_pub.key`: verejný kľúč používateľa,
- `tools/fingerprint/pqc_keys/fp_templates.json`: mapovanie používateľov na fingerprint template ID,
- `tools/fingerprint/pqc_keys/sign_counter.json`: lokálne čítače podpisov,
- `server/logs/metrics.server.jsonl`: serverové metriky,
- `tools/fingerprint/logs/metrics.agent.jsonl`: agentové metriky,
- `tools/fingerprint/biometric_probe_results.csv`: výsledky jednotlivých biometrických pokusov,
- `tools/fingerprint/frr_pilot_results.csv`: súhrnné výsledky FRR pilotu.

Hlavné procesy systému

Táto sekcia poskytuje procesný pohľad na hlavné používateľské scenáre systému. Podsekcie rozdeľujú opis na registráciu používateľa, registráciu zariadenia a samotné prihlásenie.

Registrácia používateľa

Registrácia používateľa prebieha v dvoch fázach:

1. frontend spustí OPAQUE registráciu a odošle `registrationRequest` na backend,
2. backend vráti `registrationResponse` a pripraví používateľský záznam,
3. frontend dokončí klientsku časť OPAQUE a odošle `registrationRecord`,
4. backend uloží registračný záznam a vydá scoped token `register_device`.

Registrácia zariadenia

Po registrácii hesla nasleduje registrácia zariadenia:

1. frontend zavolá lokálny agent endpoint `/pqc/register`,
2. agent pridelí používateľovi fingerprint template ID,
3. agent zaregistruje odtlačok do senzora AS608,
4. agent vygeneruje pár kľúčov ML-DSA-44,
5. súkromný kľúč zašifruje a uloží lokálne,
6. verejný kľúč odošle backendu na endpoint `/api/pqc/register`.

Prihlásenie

Prihlásenie je rozdelené na OPAQUE časť a na druhý faktor:

1. klient spustí OPAQUE `startLogin`,
2. backend vytvorí `loginResponse` a dočasne si uloží stav prihlásenia,
3. klient dokončí vlastnú časť výpočtu a odošle `finishLoginRequest`,
4. backend dokončí OPAQUE a vydá scoped token `pqc_2fa`,
5. klient požiada backend o challenge cez `/api/pqc/challenge`,
6. agent po úspešnom odtlačku podpíše challenge a vráti `payload` a `signature`,
7. backend overí payload, čítač podpisu aj samotný ML-DSA podpis,
8. po úspechu server vydá finálny JWT token.

Prehľad API rozhraní

V tejto časti sú zhrnuté komunikačné rozhrania, cez ktoré jednotlivé komponenty systému spolupracujú. Najprv sú uvedené backendové endpointy a následne rozhrania publikované lokálnym agentom.

Backendové endpointy

Hlavné backendové endpointy sú:

- `GET/api/health`,
- `POST/api/register/start`,
- `POST/api/register/finish`,
- `POST/api/pqc/register`,

- `POST/api/pqc/challenge`,
- `POST/api/pqc/verify`,
- `POST/api/login/start`,
- `POST/api/login/finish`,
- `GET/api/admin/users`.

V serveri sú prítomné aj pomocné endpointy `/api/register/biometric`, `/api/verify/biometric`, `/api/biometric/helper/:username` a proxy endpoint `/api/pqc/register-device`. Tieto sú súčasťou riešenia, ale aktuálne používateľské rozhranie ich priamo nevyužíva v hlavnom scenári.

Endpointy lokálneho agenta

Agent publikuje najmä tieto endpointy:

- `GET/health`,
- `POST/biometric/probe`,
- `POST/pqc/register`,
- `POST/pqc/sign`.

Bezpečnostné mechanizmy

Nasledujúca sekcia sumarizuje mechanizmy, ktorými riešenie chráni jednotlivé fázy autentifikačného procesu. Postupne sa venuje ochrane hesla, druhého faktora, lokálnemu uloženiu súkromného kľúča a transportnej vrstve.

Ochrana hesla

Systém využíva protokol OPAQUE, takže backend nepracuje s heslom v otvorenej forme. Server uchováva iba registračný záznam `registrationRecord` a dlhodobý parameter `server_setup`.

Ochrana druhého faktora

Druhý faktor je založený na lokálnom súkromnom kľúči a biometrickom odomknutí. Podpisovaný payload obsahuje:

- challenge od backendu,
- identifikátor relying party,
- pôvod požiadavky,
- používateľské meno,

- príznak používateľského overenia,
- časovú značku,
- monotónne rastúci čítač podpisu `signCount`.

Backend odmietne podpis v prípade expirovanej challenge, opakovaného použitia challenge alebo nezvyšujúceho sa čítača podpisu.

Lokálna ochrana súkromného kľúča

Súkromný kľúč nie je ukladaný v otvorenej podobe. Agent používa:

- AES-256-GCM na šifrovanie súkromného kľúča,
- samostatný náhodný KEK,
- device master key na ochranu KEK,
- AAD viazané na konkrétneho používateľa.

Transportná ochrana

Frontend, backend aj agent používajú HTTPS. Backend vynucuje TLS 1.3. Repozitár obsahuje lokálne certifikáty vhodné pre laboratórne a demonštračné použitie.

Monitorovanie a experimentálna podpora

Táto časť opisuje nástroje, ktoré podporujú meranie, diagnostiku a experimentálne vyhodnocovanie systému. Najprv sa zameriava na zber metrík a následne na pilotné meranie biometrickej charakteristiky FRR.

Metriky

Backend aj agent zapisujú metriky vo formáte JSON Lines. Každý záznam obsahuje najmä:

- čas začiatku a konca operácie,
- trvanie v milisekundách,
- `trace_id` a `span_id`,
- stav operácie,
- CPU a pamäťové ukazovatele.

Server okrem toho zaznamenáva HTTP cestu, metódu a status. Agent navyše eviduje aj I/O štatistiky procesu.

Pilotné meranie FRR

Skript `run_frr_pilot.py` vykonáva interaktívne meranie FRR s pevne nastavenou konfiguráciou:

- prahy zhody 20, 30 a 40,
- 20 pokusov pre každý prah,
- výstup do súboru `frr_pilot_results.csv`.

Každý jednotlivý pokus je zároveň zaznamenaný agentom do súboru `biometric_probe_results.csv`.

Známe obmedzenia implementácie

Aktuálny stav riešenia je funkčný ako laboratórny a demonštračný prototyp, avšak obsahuje viacero obmedzení:

1. dashboard je na klientovi chránený iba prítomnosťou tokenu v `localStorage`,
2. endpoint `/api/admin/users` v aktuálnej implementácii nevykonáva server-side autorizáciu,
3. stav OPAQUE prihlasovania je uložený len v pamäti backend procesu,
4. verifikácia podpisu na serveri prebieha synchronným volaním Python skriptu,
5. agent pri komunikácii s backendom používa laboratórne nastavenie `verify=False`,
6. implementácia biometrie je pevne viazaná na hardvér AS608 a lokálny sériový port,
7. repozitár obsahuje aj ukážkové databázové, certifikačné a kľúčové artefakty vhodné na reprodukovateľnosť, nie na produkčné nasadenie.

Analyzovaný systém predstavuje konzistentné demonštračné riešenie, ktoré prepája OPAQUE autentizáciu, biometrické overenie a postkvantové podpisovanie do jedného viacfaktorového scenára. Najsilnejšou stránkou riešenia je praktické prepojenie webovej aplikácie, lokálneho hardvéru a kryptografických mechanizmov v jednom funkčnom prototype. Súčasne však platí, že pred produkčným použitím by bolo potrebné doplniť robustnejšiu autorizáciu, profesionálnu správu certifikátov a silnejšie prevádzkové zabezpečenie.